UNIVERSITY OF SCIENCE – VNUHCM
FACULTY OF INFORMATION TECHNOLOGY

Project Proposal

Introduction to Software Engineering (CSC13002)

Lecturers in charge: Trần Duy Hoàng, PhD
Phạm Hoàng Hải, MSc
Trương Phước Lộc, MSc

Table of Contents

List of Tables

List of Figures

1 Member Contribution Assessment

ID	Name	Contribution (%)	Signature
24127540	Lê Tấn Thành	100%	
24127234	Lê Song Anh Tài	100%	
23127008	Nguyễn Trọng Tài	100%	
24127479	Nguyễn Bách Nhuận	95%	

Table 1. Membership and Contribution in Project proposal

2 Preliminary Problem Statement

2.1 Business Description

Nowadays, individuals and small businesses commonly publish content on more than one social media platform in order to reach their audience wherever they are - most commonly on Facebook and LinkedIn. However, each platform has its own posting interface, login process, and formatting rules, forcing users to repeat the same upload process separately on each one. For a business with multiple people involved in content creation, this problem is compounded: a content creator may produce a post, but a manager still needs a way to review and approve it before it goes live, and there is currently no single place to do this across both platforms at once.

Motivated by this problem, we propose to design and build a web-based platform that allows users to manage and distribute content to their Facebook and LinkedIn accounts from a single, unified interface, while also supporting a review-and-approval workflow for businesses with more than one person involved in publishing.

The platform is intended for two main types of end users: Individual users, who manage their own social media presence alone, and Business users, where multiple staff members are involved in producing and publishing content. To support the Business case, the system provides two account roles:

- **Manager (Business)**: Acts as the manager of a team. A Manager creates a Workspace, invites Members to join it, connects the business's Facebook and/or LinkedIn accounts, and is responsible for reviewing draft content submitted by Members and deciding whether it can be published. A Manager has access to Dashboard, Statistics, Post Management, Distribution, and Team Management features.
- **Member (Business)**: A subordinate role used by staff within a business. A Member can compose content and submit it for the Manager's review but cannot publish directly. A Member has access to Dashboard, My Analytics, My Tasks & Calendar, and Post Management features.
- **Individual**: A user working alone who has the same feature access as a Manager - Dashboard, Statistics, Post Management, and Distribution - but without Team Management, since there is no team to manage. An Individual reviews and publishes their own content directly, without needing approval from anyone else.

To use the system, a user must first register an account, choosing whether they are registering as an Individual or as a Business (in which case they become the Manager of that business and may later invite Members to join their Workspace). The Manager (or Individual) must connect their Facebook and/or LinkedIn account by authorizing the platform through each platform's official login mechanism.

Any user able to create content can compose a new post by providing a caption and a media attachment (an image), and select to which connected platforms (Facebook, LinkedIn, or both)

the post should be published. If the post is created by a Member within a Business, it is saved as a draft pending review; the Manager can then view all pending drafts and approve (allowing the post to be published) or reject (sending it back with optional feedback) each one. If the post is created by an Individual, or by a Manager directly, no approval step is required.

Once approved (or created directly by a Manager/Individual), a post can be published immediately. Any post that is still in draft or pending review can be viewed, edited, or cancelled by users with the appropriate permission. Once published, the post's status is shown per platform (e.g., posted successfully or failed, with a basic reason for failure such as expired authorization).

The system also keeps a history of posts, viewable by all users with permission, filterable by platform, status (draft, pending review, rejected, scheduled, published, failed), or, within a Workspace, by which Member submitted it. A Manager can disconnect a connected Facebook or LinkedIn account at any time, after which the platform can no longer publish on the business's behalf for that platform.

2.2 Operating Environment

The system is delivered as a web application, accessible through any modern web browser supporting HTML5, CSS3, and JavaScript (e.g., Chrome, Firefox, Edge), with no additional client-side installation required. A pure web-based delivery removes the need for users to install a separate desktop or mobile application, allowing Managers, Members, and Individuals to access the platform from any device with a browser - matching the target users (small business staff and individual creators) who expect low-friction, anywhere access.

The front-end runs entirely in the browser and communicates with the back-end through a RESTful API over HTTPS. HTTPS is required because the system transmits sensitive data - login credentials and OAuth access tokens - which must be protected in transit; a RESTful API keeps the front-end and back-end decoupled so each can be developed and scaled independently.

The back-end runs as a server-side application exposing RESTful endpoints, served through an ASGI server (e.g., Uvicorn). An ASGI-based server supports asynchronous request handling, which benefits I/O-bound operations such as calling the Facebook and LinkedIn APIs and handling scheduled publishing without blocking other requests.

To optimize workflow efficiency and isolate third-party integration overhead, the back-end architecture delegates multi-channel communication tasks to a self-hosted instance of n8n. Running alongside the primary FastAPI application server, this dedicated automation engine functions as an asynchronous microservice responsible for handling data payload mapping, retry logic, and multi-platform webhook events. When a post status changes to approved, the FastAPI server dispatches a structured webhook payload to the n8n gateway. The n8n engine then completes the distribution cycle asynchronously, ensuring that communication bottlenecks with external social networks never degrade the responsiveness of the core web server.

The system communicates with external platforms exclusively through their official public APIs - the Facebook Graph API and the LinkedIn API (Marketing/Share API) - for authentication (OAuth 2.0) and content publishing. Using each platform's official API is the only legitimate and stable way to publish content and authenticate users on behalf of a business, and guarantees compliance with each platform's developer policies, which is mandatory for any third-party

publishing tool. Facebook is also used to cover Instagram, since Instagram business accounts are managed through the same Facebook Graph API and are already synchronized with a connected Facebook Page - meaning a separate Instagram integration is unnecessary at this stage.

The system is developed and tested in a typical academic environment (local development machines), with no production-grade deployment infrastructure required for this course project.

2.3 Design and Implementation Constraints

The system shall be implemented as a client-server web application, with a clearly separated front-end and back-end. Separating the two layers allows the front-end (React) and back-end (Python) teams to work in parallel, and keeps all business rules (such as the approval workflow) enforced in one place - the back-end - rather than duplicated on the client.

Front-end: developed using JavaScript with React and Vite as the build tool, styled with Tailwind CSS, and using React Router to enforce role-based (Manager / Member / Individual) protected routes. React's component model is well suited to building the visually similar but functionally different Dashboards (Manager, Member, Individual) by composing and reusing shared components; Vite is chosen over older bundlers for its fast startup and hot-module-reload, speeding up iteration within the course timeline; Tailwind CSS keeps styling consistent across the many role-specific screens without writing large amounts of custom CSS.

Back-end: developed using Python, with FastAPI (or Django REST Framework) as the web framework, following a RESTful API architecture. Python is chosen per the team's existing skill set; FastAPI in particular is chosen for its native support for asynchronous I/O (useful for calling external platform APIs and scheduling tasks), automatic request/response validation via Pydantic, and auto-generated OpenAPI documentation, reducing documentation overhead for the team. To enforce a decoupled design, the system restricts FastAPI to handling core business rules, user states, and authorization layers. The platform integrates a self-hosted n8n instance to manage external connection states and automate the publishing pipelines to Meta and LinkedIn platforms. Furthermore, to secure access tokens obtained via the OAuth 2.0 protocol, the storage encryption mechanism applies strictly to credentials handled across both the relational database and the n8n automation ecosystem.

The system shall use a relational database - PostgreSQL - to store users, businesses/workspaces, connected accounts, posts (including their review status), and schedules. The data model is inherently relational - a Workspace has many Members, a Member's Post must reference a review status controlled by a Manager, and Posts reference connected Platform Accounts - so a relational database enforces this integrity through foreign keys and transactions more reliably than a NoSQL alternative. PostgreSQL is chosen specifically because it is free and open-source, and its json type allows flexible storage of platform-specific metadata (e.g., differing fields returned by Facebook vs. LinkedIn) without abandoning the relational model.

Login to the platform itself shall use a standard, secure authentication method: hashed passwords combined with JWT-based session tokens, transmitted only over HTTPS. Password hashing protects user credentials even if the database is compromised; JWTs are chosen because they are stateless and can carry the user's role (Manager/Member/Individual) directly in the token payload, simplifying role-based authorization checks on the back-end.

The system shall enforce role-based access control (RBAC), ensuring only Managers (or Individuals acting on their own content) can approve and publish posts, and that each role can only access the features assigned to it (as defined in Section 2). Since the core business rule of the system is that a Member's content must be approved before publishing, this check must be enforced at the back-end level - not just hidden in the UI - otherwise a Member could bypass the approval workflow by calling the API directly.

Connecting Facebook/LinkedIn accounts shall use the OAuth 2.0 protocol as provided by each platform. OAuth 2.0 is the mandated authorization protocol for third-party apps on both Meta and LinkedIn; it also means the platform never needs to see or store the user's actual Facebook/LinkedIn password, reducing security liability.

Access tokens obtained from Facebook/LinkedIn must be stored securely (e.g., encrypted at rest) and must never be displayed in plain text anywhere in the user interface. These tokens grant publishing access to a business's social accounts; if leaked, an attacker could post content or access data on the business's behalf, so they are treated with the same sensitivity as a password.

All source code shall be version-controlled using Git/GitHub, and the API shall be documented using Swagger/OpenAPI. Version control is necessary for team collaboration and tracking changes across the project timeline; auto-generated Swagger/OpenAPI documentation reduces manual documentation effort and gives graders/reviewers an interactive way to inspect and test the API.

As this is a school-based project, no production deployment infrastructure (e.g., load balancer, CDN, cloud hosting) is guaranteed to be delivered; the system only needs to run correctly in a development/demo environment. The project is scoped and graded as an academic exercise in design and implementation, not as a production service, so infrastructure concerns like scalability and high availability are explicitly out of scope. Including the React/Vite front-end, FastAPI back-end, PostgreSQL database, and the self-hosted n8n automation node - shall be containerized and orchestrated locally using a unified docker-compose.yml environment.

3 Proposed Solution

3.1 Software

3.1.1 Features

ID	Feature	The user, the goal	The benefit	Priority	Release
F01	Prompt and Context Management	As a manager, member, or individual, I want to create, update, delete, and organize AI prompt templates and context data.	So that I can streamline content generation and reuse prepared prompts for automated content.	5	1
F02	Main Dashboard and Statistics	As a manager, member, or individual, I want to view a main dashboard with system modules and statistics.	So that I can manage content activities and monitor the platform from one centralized interface.	5	1
F03	Authorization	As a manager, member, or individual, I want to access system functions according to my assigned role.	So that the platform can separate permissions between managers, members, and individuals.	5	1
F04	Post Management	As a member or individual, I want to create, draft, edit, organize, and save generated posts.	So that I can manage all created content before distributing it to social platforms.	5	1
F05	Social Platform Distribution	As a manager, member, or individual, I want to publish finalized posts to connected social media platforms.	So that my approved content can be distributed successfully across selected channels.	5	2
F06	Review and Approve Post	As a manager, I want to review and approve	So that published content meets	4	2

		generated posts before publication.	workflow control and quality requirements.		
F07	Optimize GEO/SEO	As a member or individual, I want to optimize my social media posts for search and generative engines responses.	So that my content can gain higher visibility, reach a wider audience, and compete more effectively in the new generative search environment.	3	3
F08	Source-grounded Generative Response Support	As a member or individual, I want to receive cited and source-grounded responses.	So that I can verify the information provided by the generative engine.	3	3
F09	Interaction Analysis	As a manager, member (Business) or individual, I want to analyze interactions related to distributed content.	So that I can understand content performance and improve future content strategy.	3	3

Table 2. Main features covered in the project

3.1.2 Software Architecture

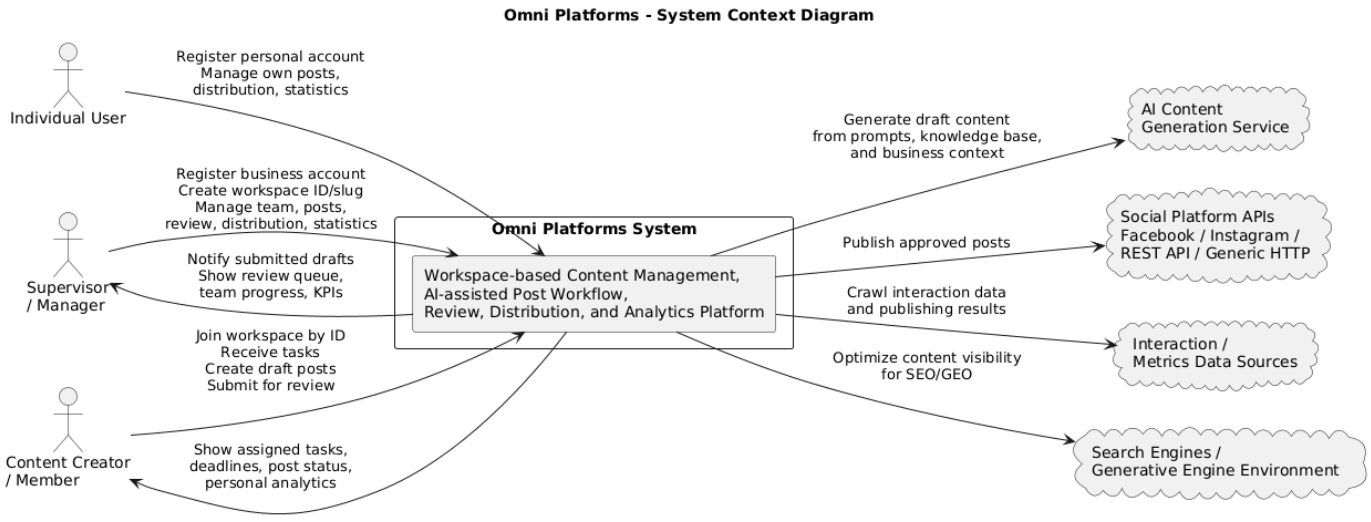

Figure 1. System Context Diagram of Omni Platforms

Omni Platforms is designed as a web-based content management and distribution platform for small and medium-sized businesses and content creators. The system provides a centralized dashboard where users can manage AI prompts and context data, create and manage generated

posts, review and approve posts, distribute finalized content to social platforms, and view basic statistics.

The system follows a layered web application architecture consisting of a web frontend, backend API server, application modules, database, and external services. The frontend provides the user interface for content creators and managers. The backend API server handles business logic, authorization, data validation, content workflow, and integration with external services. The database stores users, roles, prompts, contexts, posts, workflow statuses, publishing results, and interaction data.

External systems include AI content generation services and social platform APIs such as Facebook and LinkedIn. The AI service supports automated content generation based on prepared prompts and context data. The social platform APIs are used to publish approved posts to selected channels.

3.1.2.1 Main Architectural Components

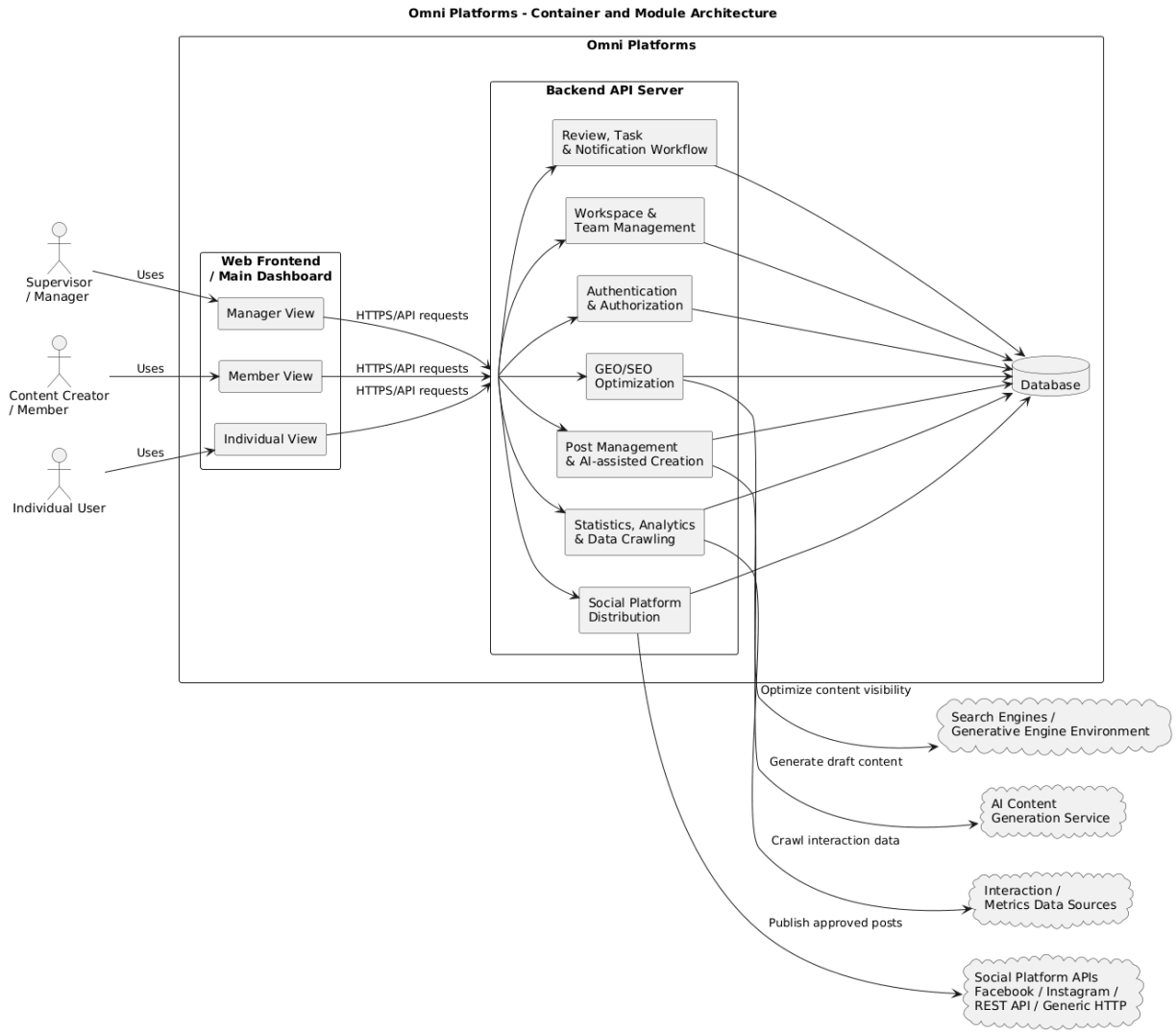

Figure 2. Container and Module Architecture of Omni Platforms

a. Web Frontend / Main Dashboard

The Web Frontend is the user-facing layer of Omni Platforms, providing a centralized entry point to the system's core features. To accommodate different user workflows, the frontend dynamically renders three role-specific interfaces: Manager View, Member View, and Individual View. Members use their dedicated dashboard to track tasks, view personal analytics, and manage content pipelines; Managers utilize it to monitor team progress, assess KPIs, and handle approvals; while Individual users access a comprehensive view to manage their standalone content lifecycle

Depending on the authorized user role, the frontend systematically includes and displays the following main interface modules:

- Main Dashboard and Statistics
- Prompt and Context Management
- Post Management & AI-assisted Creation
- Review and Approval (Manager View only)
- Team Management (Manager View only)
- Social Platform Distribution
- Optimize GEO/SEO
- Interaction Analysis

b. Backend API Server

The Backend API Server (powered by FastAPI) is responsible for processing incoming HTTPS requests from the web frontend, enforcing business logic, and coordinating core system modules. It systematically validates user actions, checks role-based authorization parameters, manages the multi-state content workflow, handles secure database persistence, and coordinates asynchronous communications with external microservices and automated engines.

To support all system functionalities across Business and Individual user branches, the backend exposes secure RESTful API endpoints for:

- User authentication and role-based authorization (via JWT over HTTPS).
- Workspace creation, member invitation, and team management tracking (Business branch only).
- Prompt and context CRUD operations within the Post Management layer.
- Post creation, multi-platform media asset processing, and status management.
- Manager review, comment logging, and content approval workflows (Business branch only).
- Webhook orchestration and data payload dispatching to the self-hosted n8n automation node for social platform publishing (Facebook and LinkedIn).
- GEO/SEO optimization metadata analysis and support.
- Interaction data processing, metric log collection, and centralized statistics generation.

c. Authorization Module

The Authorization Module enforces Role-Based Access Control (RBAC) and controls system access based on user account types and roles. The platform distinguishes between two operational branches: Business(consisting of Manager and Member roles) and Individual users. Within a business workspace, a Member is permitted to manage prompts, contexts, and draft content, while a Manager holds exclusive privileges to review, edit, and approve those drafts before publication. Individual users, operating alone, possess full administrative rights over their own content lifecycle without requiring external verification.

This module securely validates session tokens (JWT) over HTTPS to ensure that users can only invoke API endpoints and frontend routes explicitly allowed for their designated role. Furthermore, it protects all publication-related actions within the backend layer, strictly ensuring that only verified, approved, or direct authorized content requests can be dispatched to external social platforms.

d. Prompt and Context Management Module

The Prompt and Context Management Module enables Members, Managers, and Individual users to create, read, update, delete (CRUD), and systematically organize AI prompt templates and organization-specific knowledge bases (contexts). Functioning as a foundational asset framework within the Post Management & AI-assisted Creation layer, these templates and contexts are actively utilized to ground and guide automated content generation workflows.

This module securely processes and stores prompt titles, context descriptions, structural tags, and associated metadata within the central relational database. It provides standardized, reusable input data definitions that feed directly into the AI content generation pipeline, ensuring that all generated drafts remain aligned with business guidelines or personal strategies

e. Post Management Module

The Post Management Module (comprising Post Management & AI-assisted Creation) enables Members and Individual users to systematically generate, draft, edit, organize, and save multimedia social media posts. Depending on the active user branch and lifecycle stage, a post can dynamically transition through a series of defined workflow statuses, including Draft (which can be tagged as "By AI"), Pending Review, Rejected, Ready for Distribution, Published, or Failed.

This module serves as the primary interface integrated with the external AI Content Generation Service. When a user selects a pre-configured prompt template and organizational knowledge base (context), the backend dispatches these grounding metrics to the generative engine. The resulting AI-generated draft is then instantly returned to the rich-text post editor, allowing the user to review, modify, apply GEO/SEO optimizations, and assign target distribution channels before further progression.

f. Review and Approval Module

The Review and Approval Module enables Managers to systematically evaluate and audit generated posts within a business workspace before they are dispatched to external networks. Under the enterprise workflow control, when a Member finishes drafting a post, the system automatically flags it and transitions its state to Pending Review, locking it from direct distribution.

The Manager can then access the centralized review queue to inspect the post content, media attachments, and selected target platforms. This module provides the Manager with two operational pathways: they can either leave feedback and mark the post as Rejected (sending it back to the Member for revision), or apply final edits and Approve the post, which updates its status to Ready for Distribution. This module serves as the primary gateway for quality assurance, branding compliance, and multi-channel publication security.

g. Social Platform Distribution Module

The Social Platform Distribution Module is responsible for publishing finalized posts to connected channels, specifically Facebook and LinkedIn, through external platform APIs. Before any distribution cycle can be triggered, the post must be transitioned into the Ready for Distribution status.

To optimize core application performance and avoid blocking I/O operations, the backend server does not communicate with external APIs directly; instead, it dispatches a structured webhook payload containing the post data to a self-hosted n8n automation engine. The n8n node asynchronously handles the data payload mapping, executes the OAuth 2.0 authorized publishing requests via the official Facebook Graph API and LinkedIn API, and manages connection retry logic.

Upon receiving the execution callback from n8n, the module processes the publishing results and updates the database states. If the external API requests succeed, the system stores the Published status alongside the returned live platform links. If an API request fails (e.g., due to expired OAuth tokens or rate limits), the module captures the platform-side error messages, records the failure, and flags the post status as Failed to notify the user.

h. Interaction Analysis Module

The Interaction Analysis Module is responsible for tracking, collecting, and analyzing user engagement metrics related to distributed content across connected channels. Classified as a nice-to-have analytical feature within the system architecture, this module triggers after successful multi-platform publication to crawl performance indicators.

It processes raw platform telemetry - such as total views, access logs, and active interactions (likes, comments, shares) - and aggregates them into the database. This module delivers structured, visual performance statistics via dedicated dashboards, enabling Managers, Members, and Individual users to evaluate content strategy effectiveness, monitor audience reach, and optimize future cross-channel publishing campaigns.

i. Database

The Interaction Analysis Module supports the analysis of interaction data related to distributed content. This module is also included as a nice-to-have feature. It can provide statistics for managers and content creators to evaluate content performance.

j. Database

The Database stores the persistent data layer required by Omni Platforms to maintain system integrity and state compliance. It acts as the centralized storage holding:

- User accounts, encrypted credentials, and role-based assignments (Manager, Member, or Individual).
- AI prompt templates, structural tags, and associated creation metadata.

- Organizational knowledge bases and business context files used for grounding generative AI models.
- Generated posts, text captions, and multimedia file reference paths.
- Post workflow statuses tracking specific content lifecycles (Draft, Pending Review, Rejected, Ready for Distribution, Published, or Failed).
- Review and approval logs, including audit trails, review timestamps, and manager feedback comments.
- Asynchronous publishing execution results, platform telemetry data, and system statistics.
- Securely encrypted external API access tokens (OAuth 2.0 credentials) and verified live social platform links

j. Workspace & team management module

The Workspace & Team Management Module serves as the primary administrative layer dedicated exclusively to the Business branch of the platform. It provides a multi-tenant workspace architecture where a Manager can initialize an enterprise environment using a unique Workspace ID and custom slug, orchestrate their content production team, assign operational tasks, and monitor collective productivity indicators (KPIs). To maintain system efficiency and clear route separation, this entire module and its associated interfaces are completely omitted for Individual users, who operate within a standalone content lifestyle without organizational dependencies.

3.1.2.2 Runtime Flow: Creating and Publishing a Post

The main runtime flow of Omni Platforms can be divided two branch Business and Individual described as follows:

A. Business

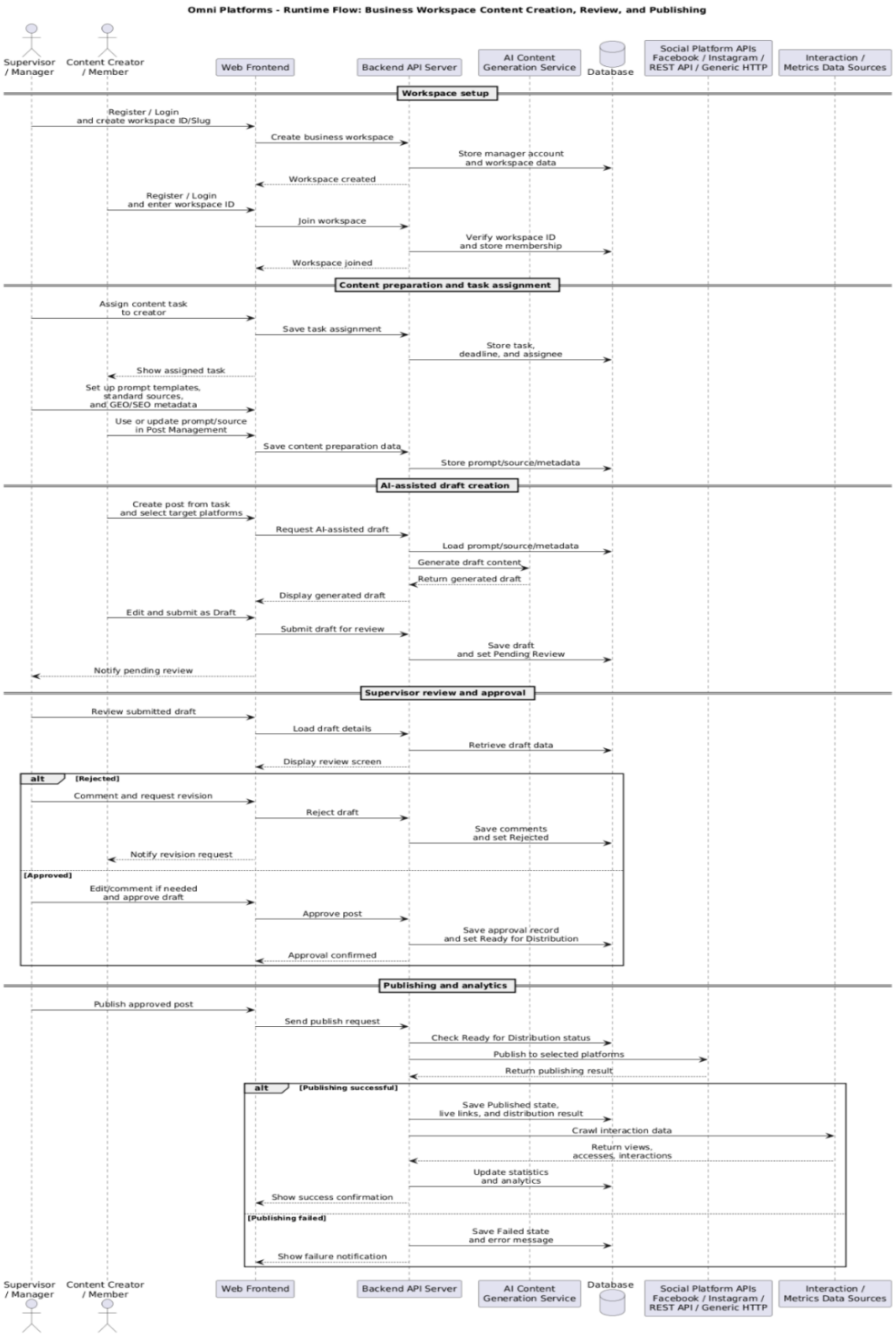

Figure 3 Runtime flow for business Workspace Content Creation, Review and Publishing

1. A content creator logs into the system
2. The system verifies the user role through the Authorization Module.
3. The Member accesses the Post Management Module to create a new post, utilizing available prompt templates and knowledge bases (contexts).
4. The Post Management Module requests AI-generated content (if requested) based on the selected prompt template.
5. The generated content is displayed in the post editor.
6. The Member edits the post and selects the target social platforms for distribution.
7. The Member submits the post, which saves it as a Draft (and marked as "By AI" if AI was utilized) to request review.
8. The Manager receives a notification, reviews the draft, makes edits or leaves comments if necessary, and approves it.
9. The approved post status is updated to Ready for Distribution.
10. The Social Platform Distribution Module automatically sends the approved post to external social platform APIs.
11. The system stores the publishing result and updates the dashboard, statistics, and team management metrics for both the Member and the Manager.

B. Individual

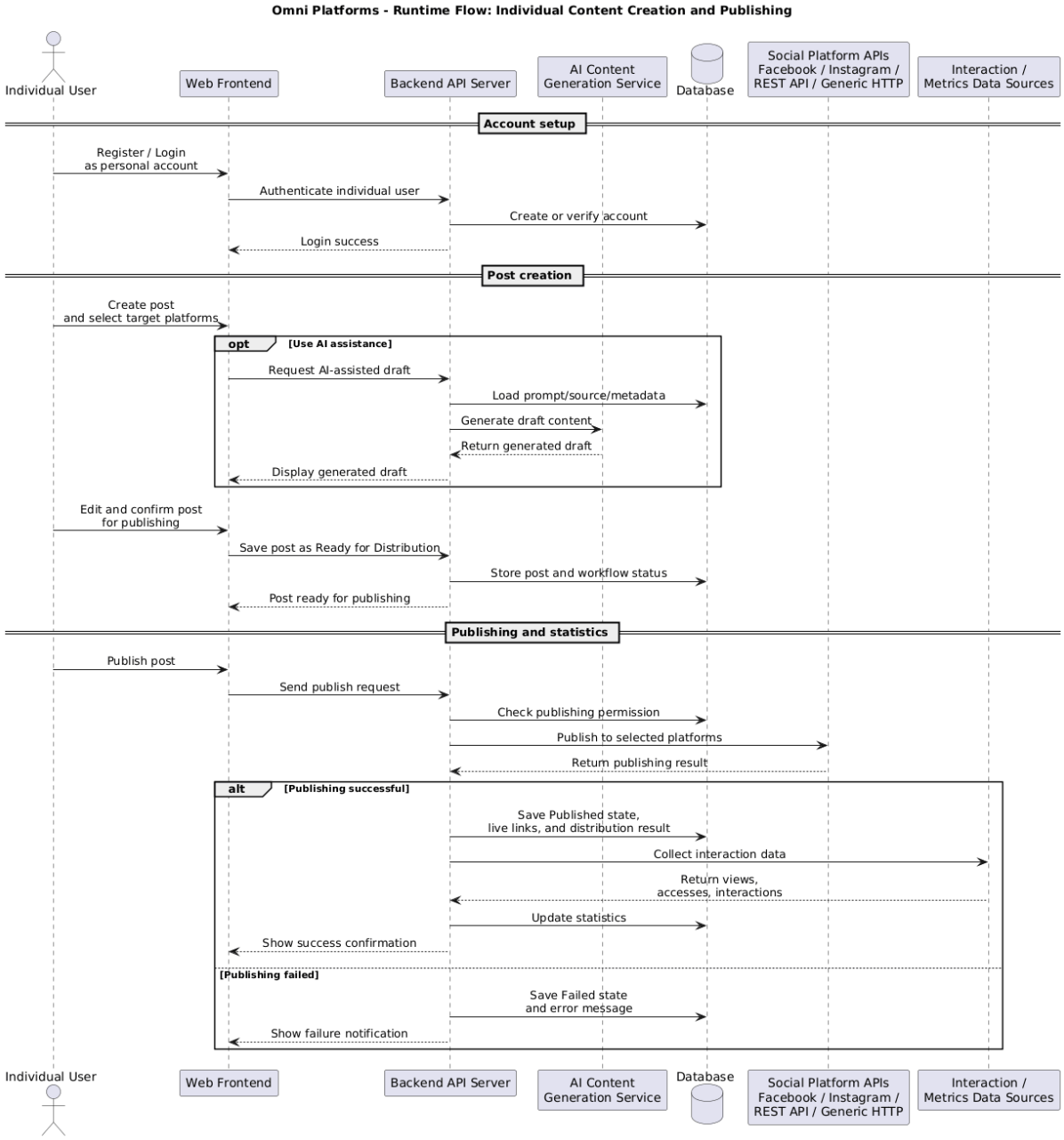

Figure 4 Runtime Flow for Individual Content Creation and Publishing

1. An individual user logs into the system.
2. The system verifies the user role through the Authorization Module.
3. The individual user accesses the Post Management Module to create a new post, utilizing available prompt templates and knowledge bases (contexts).
4. The Post Management Module requests AI-generated content (if requested) based on the selected prompt template.
5. The generated content is displayed in the post editor.
6. The individual edits the post and selects the target social platforms for distribution.

7. The individual user directly publishes the post to the selected social platforms (bypassing any review process).
8. The Social Platform Distribution Module sends the post to external social platform APIs.
9. The system stores the publishing result and updates the personal dashboard and analytics statistics.

3.1.2.3 External System Integration

Omni Platforms integrates with external services for AI content generation and social platform distribution.

The AI content generation service is used to generate draft content from prompts and contexts. The social platform APIs are used to distribute approved content to platforms such as Facebook and LinkedIn.

Since social platforms are external systems, the architecture should handle possible integration failures, including expired authorization tokens, failed API requests, and platform-side limitations. In such cases, the system should update the post status to Failed and notify the user to reconnect or re-authenticate the social platform account.

3.1.2.4 Security and Authorization Considerations

The system should apply role-based access control between content creators and managers. Sensitive operations such as approving posts and publishing to social platforms should require proper authorization.

The system should also protect social platform API credentials and access tokens. Credentials should not be exposed through the frontend. API calls to external platforms should be handled by the backend server.

3.2 Hardware

3.2.1 Development Environment Requirements

These are the hardware specifications required by the team members to design, develop, and test the web application locally.

Equipment / Device	Minimum Specifications	Recommended Specifications	Purpose
Development Laptops / PCs	CPU: Intel Core i5 / AMD Ryzen 5 RAM: 8 GB Storage: 256 GB SSD OS: Windows 10 / macOS / Linux	CPU: Intel Core i7 / Apple M-Series RAM: 16 GB Storage: 512 GB NVMe SSD OS: Windows 11 / macOS / Linux	Coding, running local web servers (Node.js/Docker), database management, and UI/UX design.

Networking Equipment	Stable Wi-Fi Connection (Minimum 15 Mbps)	High-speed Internet (50+ Mbps)	Code synchronization via Git/GitHub, online team meetings, and research.

Table 3. Device requirements

3.3.2 End-User System Requirements

The hardware and software constraints required for the target audience to access and use the website without lag.

A. Client Hardware Requirements

- Device Type: Desktop PC, Laptop
- Processor & RAM: Any standard processor with at least 2 GB of RAM (capable of running a modern web browser).
- Display Resolution: Minimum 1024x768 for desktops
- Internet Connection: Stable internet connection (3G/4G/5G or Broadband Wi-Fi) with a minimum speed of 2 Mbps.

B. Supported Web Browsers (Software Requirement for Hardware)

To ensure the website renders correctly, the user's hardware must support the latest versions of:

- Google Chrome
- Microsoft Edge
- Mozilla Firefox
- Apple Safari

4 Development Plan

4.1 Requirements Analysis

Specific Tasks:

- Gather business requirements and define project scope from the initial problem statement.
- Identify and classify Functional Requirements (FR) and Non-functional Requirements (NFR) such as security, performance, and platform constraints.
- Model system behaviors by creating Use Case diagrams and detailing Use Case specifications.
- Conduct User Story Mapping sessions to prioritize the Product Backlog, configure Epics, and establish Sprint schedules directly on the Jira Platform.

Specific Deliverables:

- Software Requirement Specification (SRS) document, Vision, and Use Cases specification.
- Initial Product Backlog Board and Sprint 1 roadmap officially configured and published on Jira.
- Storage Path: /docs/requirements/

4.2 Software Design

Specific Tasks:

- Architecture Design: Define the overall system architecture (e.g., MVC, Client-Server) and draw the system block diagram.
- Database Design: Create the Entity-Relationship Diagram (ERD) and transform it into physical database schemas.
- UI/UX Design: Sketch wireframes and design high-fidelity interactive mockups using Figma.
- Component Design: Construct Class Diagrams and Sequence Diagrams (UML) to map out internal system logic and object interactions.

Specific Deliverables:

- System Architecture Design document. Storage Path /docs/analysis_and_design/architecture/
- UML Diagrams (Class, Sequence, etc.). Storage Path: /docs/analysis_and_design/uml/
- Entity-Relationship Diagram (ERD) & Database Schema. Storage Path: /docs/analysis_and_design/database/

- UI/UX Mockups & Wireframes Link. Storage Path: /docs/analysis_and_design/ui_design/

4.3 Implementation

Specific Tasks:

- Initialize the project repository structure on GitHub exactly as configured.
- Setup the development environment and initialize the database instance.
- Develop frontend components matching the approved Figma mockups.
- Implement backend business logic, RESTful APIs, security protocols, and authentication modules.
- Integrate frontend modules with backend services.

Specific Deliverables:

- Clean, functional, and well-documented Source Code.
- Comprehensive commit history matching assigned tasks on GitHub Issues.
- Storage Path: /src/

4.4 Testing

Specific Tasks:

- Unit Testing: Write and execute isolated test components during the development phase.
- Integration Testing: Verify seamless data flow and communication between frontend, backend, and the database.
- System Testing: Execute end-to-end testing based on predefined test cases to detect logical flaws, UI bugs, or security vulnerabilities.
- Bug Fixing: Log tracking issues on GitHub Issues, assign tasks to developers, and re-test after bugs are resolved.

Specific Deliverables:

- Test Plan & Test Case Specification document.
- Defect/Bug Log Report.
- Storage Path: /docs/test/

4.5 Deployment and Maintenance

Specific Tasks:

- Build and package the application (e.g., compiling production builds or creating Docker images).
- Deploy the system onto the target production environment (Cloud platforms, VPS, or hosting servers).
- Monitor system stability, server resource utilization, and gather initial user feedback.

- Formulate a maintenance schedule, update Project_Plan.md, and log weekly progress.

Specific Deliverables:

- Live Production URL / Demo Link.
- Deployment Guide & System User Manual. Storage Path:
- /docs/management/
- Updated project status in Project_Plan.md. Storage Path: /docs/management/Project_Plan.md

5 Human Resources & Costing Plan

5.1. Overall Team Structure & Roles

ID	Name	Role
24127540	Lê Tấn Thành	Project Manager & QA Leader
24127234	Lê Song Anh Tài	UI/UX Designer & Frontend Developer
23127008	Nguyễn Trọng Tài	Tech Lead, Backend Developer & Integration QA
24127479	Nguyễn Bách Nhuận	Backend Developer & Backend QA

Table 4. Designated roles for the project

5.2 Resource Allocation per Development Phase

Phase	Key Roles	Assigned Members	Main Tasks
Phase 1: Initiation & Analysis *(Week 1 - 2)*	PM, Business Analyst (BA)	All Members	Conduct market research, gather requirements, and finalize the SRS (Software Requirement Specification) document.
Phase 2: System Design *(Week 3 - 4)*	Technical Lead, UI/UX Designer	Lê Song Anh Tài Nguyễn Trọng Tài Nguyễn Bách Nhuận	Anh Tài: Create wireframes and build UI mockups. Bách Nhuận & Trọng Tài: Design database schema and system architecture.
Phase 3: Development (Coding) *(Week 5 - 10)*	Dev Team, Unit QA	Lê Song Anh Tài Nguyễn Trọng Tài Nguyễn Bách Nhuận Lê Tấn Thành	Anh Tài, Bách Nhuận and Trọng Tài:Core programming, API development, frontend-backend connection, and internal code cross-testing.

			Tấn Thành: Facilitate weekly Sprint syncs, manage Jira board velocity, and write acceptance test cases for core modules
Phase 4: Testing & Deployment *(Week 11 - 12)*	QA/QC, PM, Dev Team	All Members	Trọng Tài: Lead system integration testing. Thành: Execute final user acceptance esting. All Members: Bug fixing and deploying to the server.

Table 5. Human resource allocation for the project

5.3 Costing plan

No.	Category	Description	Unit	Quantity	Total cost
1	Domain name	National Domain (.edu.vn)	250.000 VND/Year	1	250.000 VND
2	Virtual Private Server	Cloud VPS (1 vCPU, 2GB RAM, 25GB SSD)	50.000 VND/ Month	3	150.000 VND
3	Reserve Fund	10%			40.000 VND
				TOTAL ESTIMATED BUDGET	440.000 VND

Table 6. Estimated cost for all planned features to be delivered

Note on Budget Allocation: Although the platform is primarily containerized and orchestrated locally via docker-compose for grading and evaluation purposes, this costing plan serves as an optional contingency infrastructure fund. In the event of webhook simulation failures, port forwarding restrictions on local university networks, or the necessity for persistent public URL verification with external APIs (Meta and LinkedIn), this budget will be deployed to migrate the microservices container directly onto a public Cloud VPS.

6 Tools setup

6.1 Moodle

- **Purpose**: Act as the primary gateway for receiving official course announcements, tracking project guidelines, and submitting all final project deliverables (Project Assessments).
- **Usage**: The team leader is responsible for checking notifications daily and ensuring all submissions are uploaded punctually according to deadlines.

6.2 Discord / Communication Platform

Purpose: Utilized for rapid daily communication, technical discussions, voice meetings, and automated workflow updates among group members.

Server Configuration:

- **Webhook Category:**
  - # github-updates: A dedicated channel linked directly to our project repository. Repository activities such as commits, pull requests, issue creation, and task updates are automatically posted to this channel to keep all members informed.
- **Information Category:**
  - # resources: Used for sharing important reference materials, links, and project documentation.
  - # meeting-history: Stores recap notes, logs, or recordings from past meetings.
  - # weekly-report: Dedicated to posting weekly progress updates and status reports.
  - # announcements: Reserved for official project-wide announcements and meeting schedules.
- **Discussion Category:**
  - # general: Dedicated to overall project discussions, casual talk, and collaborative brainstorming.
  - # question: Used for technical discussions, coding issues, debugging, and implementation Q&A.
- **Voice Category:**
  - Waiting Room: A standby channel for members waiting to join meetings or discussions.

- Meeting Room: Used for online meetings, sprint updates, and official discussions.
- Pair Programming: Dedicated to live collaborative coding and technical pairing sessions.

Usage Policy: All members are required to install Discord on both desktop and mobile devices and enable notifications to ensure timely communication and prompt responses throughout the project lifecycle.

6.3 Jira / Project Management Platform

Purpose: Utilized for agile project management, task tracking, and monitoring development progress to ensure transparency and timely delivery among group members.

Board & Sprint Configuration:

- Project Board: A single, centralized Scrum board is established for the group to aggregate the product backlog and visualize active workflows.
- Sprint Management: Tasks are structured and organized into time-boxed sprints (typically 1–2 weeks) with clearly defined goals to facilitate iterative development and incremental feature delivery.
- Workflow Stages: Tasks progress sequentially through standard lifecycle columns on the board:
  - Backlog / To Do: Planned tasks, user stories, and requirements awaiting development in the current or upcoming sprints.
  - In Progress: Tasks currently assigned to and actively being worked on by team members.
  - In Review / Testing: Completed tasks undergoing peer code review, quality assurance, or integration testing.
  - Done: Fully implemented, verified, and approved tasks that meet the project's definition of done.
- Issue Categorization: Work items are classified into Epics (major system features), Stories/Tasks (individual technical requirements), and Bugs (defects or issues) for clear traceability.

Usage Policy: All members are required to update the status of their assigned tasks in real-time, estimate task efforts accurately, and ensure the Jira board reflects the exact state of progress prior to group syncs and sprint reviews.

6.4 GitHub / Version Control Platform

Purpose: Utilized for source code management, version control, technical documentation hosting, and milestone tracking to ensure code integrity and seamless collaboration among group members.

Repository Directory Structure: The root repository Software_Project_Hcmus/ is systematically structured to cleanly decouple production source code, technical design documentation, and official academic submissions:

- **src/**: Houses the primary source code and core functional implementation of the project.
- **docs/**: Centralized directory for all technical, architectural, and administrative documentation.
  - **proposal/**: Stores the official project proposal documentation (Project_Proposal.md).
  - **management/**: Contains administrative records tracking project execution and progress.
    - **Project_Plan.md**: Outlines project schedules, development timelines, and Gantt charts.
    - **weekly_reports/:** Archives routine weekly progress updates and team status reports.
    - **meeting_minutes/:** Chronologically stores team meeting notes (standardized as YYYY-MM-DD_Meeting.md).
  - **requirements/**: Contains requirement specifications including project Vision, Use Cases, and product backlogs.
  - **analysis_and_design/:** Houses system architecture layouts and design artifacts.
    - **architecture/**: Specifies the high-level structural design of the software system.
    - **uml/**: Stores all Unified Modeling Language (UML) diagrams.
    - **database/**: Contains Entity-Relationship Diagrams (ERD) and database schemas.
    - **ui_design/:** Holds UI/UX wireframes, interface layouts, and design components.
  - **test/**: Contains quality assurance assets, including software test plans and test cases.
- **pa/**: Dedicated folder for formal Project Assessments and official academic submissions.
  - **PA1/**: Deliverables, source code, and artifacts submitted for Phase 1 evaluation.
  - **PA2/**: Deliverables, source code, and artifacts submitted for Phase 2 evaluation.
  - **PA3/**: Deliverables, source code, and artifacts submitted for Phase 3 evaluation.
- **README.md**: Located at the repository root to provide a comprehensive project overview, setup instructions, and the core team Contribution Table.

Usage Policy: All members must strictly adhere to the designated branching strategy, follow uniform commit message conventions, and submit modifications through Pull Requests. Code

reviews are mandatory before merging any features into the stable branch, and repository events will trigger automated updates linked directly to the team's Discord server.